

SDEV 2351

DAY 2: MULTIPLE ROUTE PARAMETERS, NAVIGATIONMANAGER, AND
PROGRAMMATIC NAVIGATION

MODULE 1 — ROUTING & STATE

Session Objectives

1. Combine multiple route parameters with constraints in a single route template
2. Use `NavigationManager` to move users between pages programmatically
3. Build wizard-style multi-step navigation flows
4. Implement conditional navigation based on application state

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	10 min	Day 1 review + multi-param preview
Exploring	25 min	Multi-param routes, NavigationManager, wizard flows
Practicing	55 min	Three exercises (guided → independent → challenge)
Assessing	10 min	Knowledge check + exit ticket

PHASE 1 · 10 MINUTES

CONNECTING

From defining routes to navigating them in code

Day 1 Recap

Yesterday you built routes with **single parameters and constraints**:

- Nested layout chains with `@layout` and `_Imports.razor`
- Route parameters: `{id:int}`, `{name:alpha}`
- Constraint chaining: `{id:int:min(1)}`
- Constraint failure = 404 (not an exception)

Today: Combine multiple parameters in one template, then use **NavigationManager** to navigate in code.

Think multi-step checkout -- the app decides where to send you next.

PHASE 2 · 25 MINUTES

EXPLORING

Multi-param routes, NavigationManager, and wizard flows

Multiple Route Parameters

A single route template can combine several parameters:

```
@page "/products/{category:alpha}/{id:int:min(1)}"

[Parameter] public string Category { get; set; }
[Parameter] public int Id { get; set; }
```

- Each segment needs its own `[Parameter]` property
- Constraints apply independently to each parameter
- Pairs naturally with `NavigationManager` for programmatic navigation

NavigationManager

Inject it like any service -- same `@inject` pattern from SDEV 2301:

```
@inject NavigationManager Navigation

Navigation.NavigateTo("/products/electronics/42");
Navigation.NavigateTo("/external", forceLoad: true);
Navigation.NavigateTo("/step/2", replace: true);
```

CLIENT-SIDE (DEFAULT)

Fast, no server round-trip. Blazor handles it in the circuit.

FORCE LOAD

Full server round-trip. Use only for non-Blazor or external URLs.

NavigateTo() Overloads

PARAMETER	EFFECT	WHEN TO USE
<code>path only</code>	Client-side navigation	Default for all Blazor-to-Blazor navigation
<code>forceLoad: true</code>	Full page reload	Non-Blazor pages or external URLs
<code>replace: true</code>	Replaces current history entry	Back buttons in wizards, post-form redirects

Common mistake: Adding `forceLoad: true` "just in case" triggers a full server round-trip, destroying all component state and the Blazor circuit.

Wizard-Style Navigation

The app decides the next page based on user actions:

- **Checkout flow:** Shipping → Payment → Confirmation
- **Conditional paths:** Reason selection determines next step
- `NavigateTo()` in event handlers drives the flow

Declarative links (`NavLink`) for static menus. Programmatic navigation (`NavigateTo`) when code decides where or when to navigate.

PHASE 3 · 55 MINUTES

PRACTICING

Three exercises — guided, independent, challenge

Practice 1: User Registration Wizard (15 min)

Guided build -- follow along on screen

- Build a 3-step registration flow
- Inject NavigationManager into each step component
- Use `NavigateTo()` to move between registration steps
- Each step is a separate page with route parameters

Practice 2: Product Return Request Wizard (25 min)

Independent practice

- Build a return request flow with **conditional paths**
- Reason selection determines the next page:
 - Refund path vs. Exchange path
- Uses multi-param routes and conditional `NavigateTo()`
- **Success threshold:** TODOs 1-8 = objective met

15-minute checkpoint: if TODOs 1-8 are not done, focus there before moving on.

Practice 3: Multi-Step Survey Wizard (10 min)

Challenge -- for early finishers

- Implement forward navigation with page-parameter routing
- Implement back navigation with `replace: true`
- `replace: true` prevents the back button from creating duplicate history entries

Common Troubleshooting

ISSUE	CAUSE	FIX
NavigationManager is null	Missing <code>@inject</code> directive	Add <code>@inject NavigationManager</code> <code>Navigation</code>
Full page reload on navigate	Using <code>forceLoad: true</code> unnecessarily	Remove <code>forceLoad</code> for Blazor pages
Back button loops	Not using <code>replace: true</code>	Add <code>replace: true</code> for wizard back steps
Second parameter always null	Missing <code>[Parameter]</code> attribute	Decorate both properties

PHASE 4 · 10 MINUTES

ASSESSING

Summary

Today you learned to:

1. **Combine multiple route parameters** with constraints in a single route template
2. **Use `NavigationManager`** to navigate programmatically with `NavigateTo()` and its overloads
3. **Build wizard-style flows** where the application decides the next page
4. **Choose between `forceLoad`, `replace`**, and default client-side navigation

Exit Ticket

A user completes step 2 of a 3-step wizard. Write the `NavigateTo()` call to send them to step 3 at `/wizard/step/3`.

What's Next

Day 3: State Challenges and the Flux/Redux Pattern

- What happens to your variables when you navigate away?
- Three state management problems: loss, drilling, sync
- The Flux/Redux pattern -- unidirectional data flow

Pre-class reading assigned: Fluxor Quick Reference Guide (~30 min) -- linked from the Day 3 page in Brightspace. Focus: store setup, actions, reducers, the Flux/Redux data flow.